

**LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 5**



Searching

Oleh:

Muhammad Kurniawan Pasya

NIM. 2510817210026

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
MEI
2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 5

Laporan Praktikum Algoritma & Struktur Data Modul 5: Searching, ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Muhammad Kurniawan Pasya

NIM : 2510817210026

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Fauzan Ahsani
NIM. 2310817310009

Ir. Muhamamd Alkaff, S,Kom, M.Kom, Ph.D
NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN.....	2
DAFTAR ISI	3
DAFTAR GAMBAR.....	4
DAFTAR TABEL	5
SOAL.....	6
A. Source Code.....	6
B. Pembahasan dan Output Program.....	8
LINEAR SEARCH.....	8
BINARY SEARCH	12
TAUTAN GIT	17

DAFTAR GAMBAR

Gambar 1 Screenshot Output Linear Searching	8
Gambar 2 Flowchart Linear Search.....	11
Gambar 3 Screenshot Output Binary Searching.....	12
Gambar 4 Flowchart Binary Search	15

DAFTAR TABEL

Tabel 1 Source Code searching_algorithms.cpp	6
--	---

SOAL

Lakukan pengujian terhadap setiap algoritma menggunakan beberapa kondisi data berikut:

1. Data acak (random) — khusus Linear Search; Binary Search wajib diuji pada data terurut.
2. Data sudah terurut menaik (sorted ascending).
3. Data terurut menurun (sorted descending) — khusus Linear Search.

Untuk setiap kondisi data, lakukan pengujian dengan tiga skenario pencarian:

- Elemen ada di awal (best case position).
- Elemen ada di tengah.
- Elemen tidak ada dalam data (not found).

A. Source Code

Tabel 1 Source Code searching_algorithms.cpp

1	#include "searching_algorithms.h"
2	#include <chrono>
3	
4	
5	
6	using Clock = std::chrono::high_resolution_clock;
7	
8	
9	void linear_search(const std::vector<int>& data, int
	target, Metrics& m) {
10	auto start = Clock::now();
11	
12	int n = (int)data.size();
13	
14	for (int i = 0; i < n; ++i) {
15	m.comparisons++;
16	
17	if (data[i] == target) {
18	m.found_index = i;
19	break;
20	}
21	}

```
22
23     m.time_us = std::chrono::duration<double,
std::micro>(Clock::now() - start).count();
24 }
25
26
27
28 void binary_search(const std::vector<int>& data, int
target, Metrics& m) {
29     auto start = Clock::now();
30
31     int left = 0;
32     int right = (int)data.size() - 1;
33
34     while (left <= right) {
35         m.comparisons++;
36
37         int mid = left + (right - left) / 2;
38
39         if (data[mid] == target) {
40             m.found_index = mid;
41             break;
42         }
43         else if (data[mid] < target) {
44             left = mid + 1;
45         }
46         else {
47             right = mid - 1;
48         }
49     }
50
51     m.time_us = std::chrono::duration<double,
std::micro>(Clock::now() - start).count();
52 }
```

B. Pembahasan dan Output Program

LINEAR SEARCH

Implementasi Linear Search dibuat menggunakan perulangan `for` yang memeriksa elemen array satu per satu mulai dari indeks pertama hingga target ditemukan. Pada setiap iterasi, program melakukan `comparison` (perbandingan) antara elemen array dengan target yang dicari. Jika nilai elemen sama dengan target, maka indeks disimpan ke dalam `found_index` dan proses pencarian dihentikan menggunakan `break`.

Tujuan penggunaan `break` adalah agar program tidak melakukan pengecekan yang tidak diperlukan setelah data ditemukan. Dengan [cara ini](#), jumlah `comparison` dapat dikurangi dan waktu eksekusi menjadi lebih cepat pada kondisi tertentu.

Cara kerja tersebut menunjukkan bahwa performa Linear Search sangat dipengaruhi oleh posisi target di dalam array. Jika target berada di awal array, program hanya membutuhkan satu `comparison` untuk menemukan data. Sebaliknya, jika target berada di akhir atau bahkan tidak ada di dalam array, maka seluruh elemen harus diperiksa satu per satu. Oleh karena itu digunakan tiga skenario pengujian yaitu `best case`, `average case`, dan `worst case` agar dapat mengetahui pengaruh posisi target terhadap performa algoritma.

```
>> Linear Search
```

Algorithm	Case	N	Comparisons	Found Index	Time(us)
Linear Search	Best	1000	1	0	5.000
Linear Search	Average	1000	375	374	3.000
Linear Search	Worst	1000	1000	Not Found	7.000
Linear Search	Best	10000	1	0	0.000
Linear Search	Average	10000	3746	3745	31.000
Linear Search	Worst	10000	10000	Not Found	128.000
Linear Search	Best	100000	1	0	0.000
Linear Search	Average	100000	37455	37454	195.000
Linear Search	Worst	100000	100000	Not Found	760.000
Linear Search	Best	1000000	1	0	1.000
Linear Search	Average	1000000	374541	374540	2774.000
Linear Search	Worst	1000000	1000000	Not Found	5751.000

Gambar 1 Screenshot Output Linear Searching

Pada best case, target diletakkan di indeks pertama sehingga algoritma langsung menemukan data pada comparison pertama. Berdasarkan hasil pengujian, untuk semua ukuran data jumlah comparison tetap bernilai 1. Hal ini menunjukkan bahwa waktu yang dibutuhkan algoritma sangat kecil karena proses pencarian langsung berhenti setelah pemeriksaan pertama. Kondisi ini memiliki kompleksitas waktu $O(1)$ karena jumlah operasinya konstan dan tidak dipengaruhi oleh ukuran data.

Pada average case, target diletakkan di posisi acak atau mendekati tengah array. Dalam kondisi ini, program perlu memeriksa sebagian elemen sebelum target ditemukan. Misalnya pada data berukuran 100000, jumlah comparison mencapai sekitar 37455. Hal ini menunjukkan bahwa semakin jauh posisi target dari awal array, semakin banyak operasi yang harus dilakukan. Karena jumlah comparison bertambah seiring bertambahnya ukuran data, maka kompleksitas waktunya termasuk $O(n)$. Dari hasil eksperimen didapatkan bahwa waktu eksekusi juga ikut meningkat ketika ukuran data semakin besar.

Pada worst case, target sengaja dibuat tidak ada di dalam array sehingga algoritma harus memeriksa seluruh elemen hingga akhir. Contohnya pada data berukuran 1000000, jumlah comparison mencapai 1000000 dengan hasil `Not Found`. Kondisi ini merupakan kondisi paling lambat karena tidak ada proses yang dapat menghentikan loop lebih awal. Oleh sebab itu, kompleksitas waktu pada worst case juga termasuk $O(n)$. Hasil pengujian menunjukkan bahwa waktu eksekusi meningkat cukup signifikan dibanding best case maupun average case.

Dari sisi penggunaan memori, Linear Search memiliki kompleksitas ruang $O(1)$ karena algoritma hanya menggunakan beberapa variabel tambahan seperti indeks loop, target, dan object metrics. Algoritma tidak membuat array baru ataupun struktur data tambahan besar selama proses pencarian berlangsung. Karena bekerja langsung pada array asli tanpa membutuhkan memori tambahan yang besar, maka Linear Search termasuk algoritma in-place.

Karakteristik utama Linear Search adalah mekanisme pencariannya yang dilakukan secara berurutan dari awal hingga akhir array. Algoritma ini hanya menggunakan proses comparison satu per satu. Linear Search juga fleksibel karena dapat digunakan pada data acak, ascending, maupun descending.

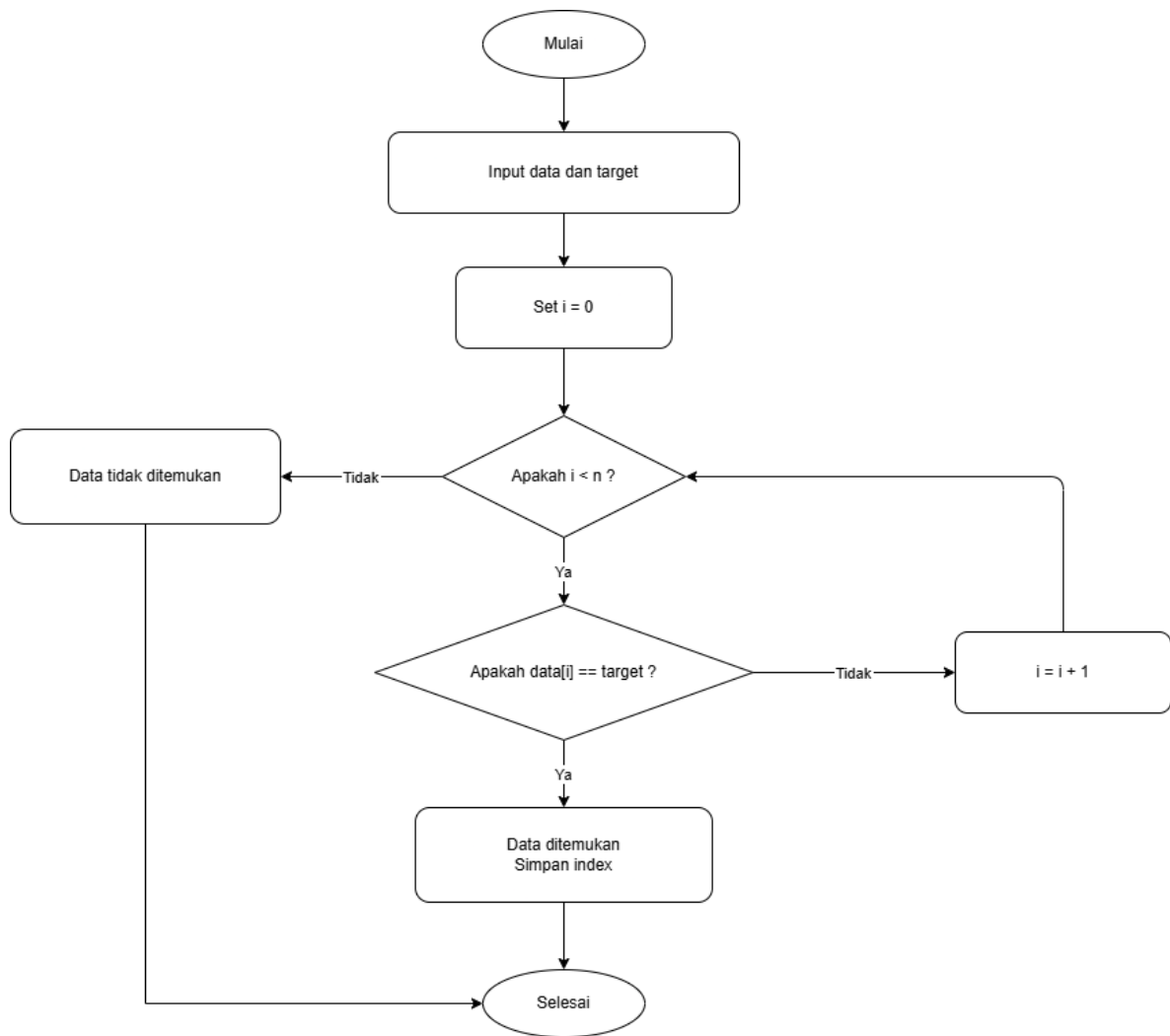
Berdasarkan analisis operasi, jumlah comparison menjadi faktor utama yang mempengaruhi performa algoritma.

Pada best case, comparison hanya bernilai 1 karena target langsung ditemukan. Pada average case, comparison meningkat sesuai posisi target di dalam array. Sedangkan pada worst case, comparison mencapai sebesar ukuran data karena seluruh elemen diperiksa. Pada algoritma ini Linear Search hanya melakukan pencarian dan tidak mengubah posisi data. Selain itu, implementasi yang digunakan bersifat iteratif sehingga tidak terdapat recursion call. Jumlah array accessed juga mengikuti jumlah comparison karena setiap comparison membutuhkan akses ke elemen array.

Dari hasil output didapatkan bahwa ukuran data sangat mempengaruhi performa Linear Search. Semakin besar nilai N , semakin besar pula jumlah comparison dan waktu eksekusinya. Misalnya pada worst case, waktu eksekusi meningkat dari 7 mikrosecond pada data berukuran 1000 menjadi 5751 mikrosecond pada data berukuran 1000000. Hal ini menunjukkan hubungan linear antara ukuran data dan jumlah operasi yang dilakukan algoritma.

Selain itu, distribusi data tidak terlalu mempengaruhi Linear Search karena algoritma tetap memeriksa elemen satu per satu. Faktor yang paling mempengaruhi performa adalah posisi target di dalam array.

Pada best case, jumlah operasi sangat kecil dan bersifat konstan sehingga sesuai dengan teori $O(1)$. Sementara itu pada average case dan worst case, jumlah comparison meningkat seiring ukuran data sehingga sesuai dengan teori $O(n)$. Dari hasil tersebut dapat disimpulkan bahwa Linear Search cocok digunakan pada dataset kecil atau ketika data belum terurut, tetapi kurang efisien untuk dataset besar karena harus memeriksa elemen satu per satu hingga target ditemukan.



Gambar 2 Flowchart Linear Search

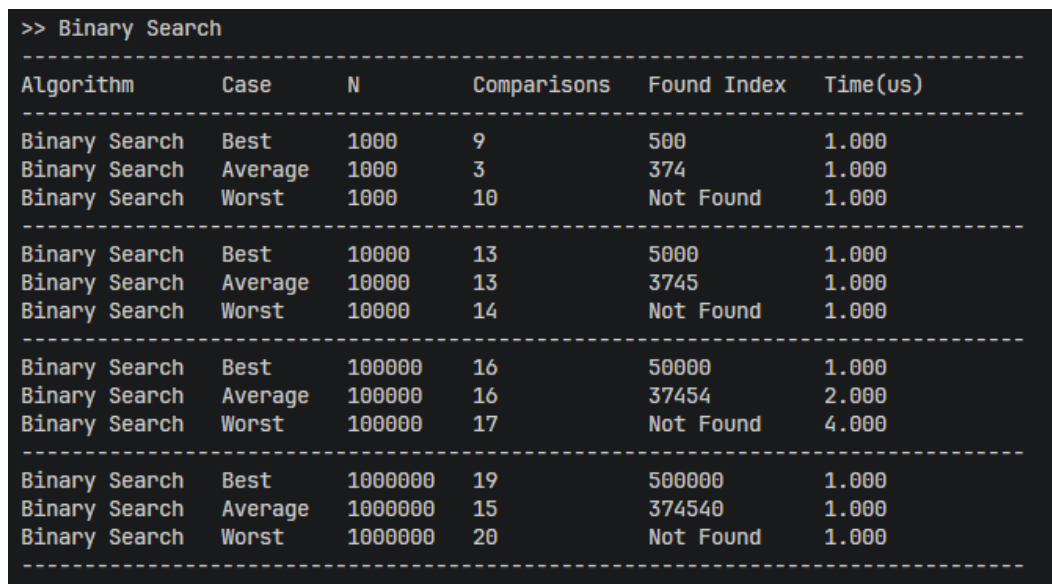
BINARY SEARCH

Implementasi Binary Search bekerja dengan cara membagi area pencarian menjadi dua bagian secara terus menerus. Karena itu, algoritma ini hanya dapat digunakan pada data yang sudah terurut secara ascending. Pada implementasi program, proses pencarian dimulai dengan menentukan indeks kiri (*left*) dan kanan (*right*). Setelah itu program mencari indeks tengah (*mid*) menggunakan rumus:

```
int mid = left + (right - left) / 2;
```

Setelah indeks tengah diperoleh, program membandingkan nilai tengah dengan target yang dicari. Jika nilai tengah sama dengan target, maka pencarian selesai dan indeks disimpan ke dalam *found_index*. Jika target lebih kecil dari nilai tengah, maka pencarian dipindahkan ke bagian kiri array. Sebaliknya, jika target lebih besar, maka pencarian dilanjutkan ke bagian kanan array.

Tujuan utama mekanisme ini adalah mengurangi jumlah data yang diperiksa pada setiap iterasi. Pada setiap comparison, area pencarian langsung dipotong menjadi setengah.



>> Binary Search					
Algorithm	Case	N	Comparisons	Found Index	Time(us)
Binary Search	Best	1000	9	500	1.000
Binary Search	Average	1000	3	374	1.000
Binary Search	Worst	1000	10	Not Found	1.000
Binary Search	Best	10000	13	5000	1.000
Binary Search	Average	10000	13	3745	1.000
Binary Search	Worst	10000	14	Not Found	1.000
Binary Search	Best	100000	16	50000	1.000
Binary Search	Average	100000	16	37454	2.000
Binary Search	Worst	100000	17	Not Found	4.000
Binary Search	Best	1000000	19	500000	1.000
Binary Search	Average	1000000	15	374540	1.000
Binary Search	Worst	1000000	20	Not Found	1.000

Gambar 3 Screenshot Output Binary Searching

Pada pengujian best case, target sengaja diletakkan pada posisi tengah array agar langsung ditemukan pada pemeriksaan pertama. Tapi pada hasil eksperimen jumlah comparison pada best case tidak selalu bernilai 1.

Contohnya pada data berukuran 1000, comparison bernilai 9. Hal ini terjadi karena target yang digunakan berada pada indeks 500, sedangkan proses pembagian area pencarian tidak selalu langsung menghasilkan indeks tersebut pada iterasi pertama. Program tetap harus melakukan beberapa kali pembagian hingga menemukan posisi target.

Pada average case, target berada pada posisi acak di dalam array. Program akan terus membagi area pencarian menjadi dua hingga target ditemukan. Dari hasil eksperimen didapatkan bahwa jumlah comparison relatif kecil meskipun ukuran data sangat besar. Contohnya pada data berukuran 1000000, average case hanya membutuhkan sekitar 15 comparison. Hal ini berarti bahwa penambahan ukuran data tidak terlalu mempengaruhi jumlah operasi Binary Search.

Pada worst case, target sengaja dibuat tidak ada di dalam array sehingga program terus membagi area pencarian sampai interval pencarian habis. Akibatnya `found_index` tetap bernilai -1 dan ditampilkan sebagai "Not Found".

Walaupun target tidak ditemukan, jumlah comparison tetap kecil. Contohnya pada data berukuran 1000000, worst case hanya membutuhkan sekitar 20 comparison. Hal ini berarti Binary Search tetap efisien bahkan ketika data tidak ditemukan.

Karena setiap iterasi selalu membagi area pencarian menjadi dua. Artinya jumlah operasi tidak bertambah secara linear mengikuti ukuran data. Sehingga kompleksitas waktunya adalah $O(\log n)$ pada average dan worst case.

Pada kondisi terbaik, Binary Search memiliki kompleksitas $O(1)$ jika target langsung ditemukan pada posisi tengah.

Algoritma hanya menggunakan beberapa variabel tambahan seperti `left`, `right`, dan `mid`. Program tidak membuat array baru maupun struktur data tambahan besar selama proses pencarian berlangsung. Oleh sebab itu, Binary Search termasuk algoritma in-place karena bekerja langsung pada array asli tanpa membutuhkan memori tambahan yang signifikan. Artinya kompleksitas ruangnya adalah $O(1)$.

Karakteristik utama Binary Search adalah mekanisme pembagian area pencarian secara berulang. Berbeda dengan Linear Search yang memeriksa data satu per satu, Binary Search langsung menghilangkan setengah bagian data pada setiap iterasi. Hal inilah yang membuat jumlah comparison menjadi sangat kecil.

Namun Binary Search memiliki batasan penting, yaitu hanya dapat digunakan pada data yang sudah terurut ascending. Jika data tidak terurut, hasil pencarian dapat menjadi salah karena proses pembagian area pencarian tidak lagi valid.

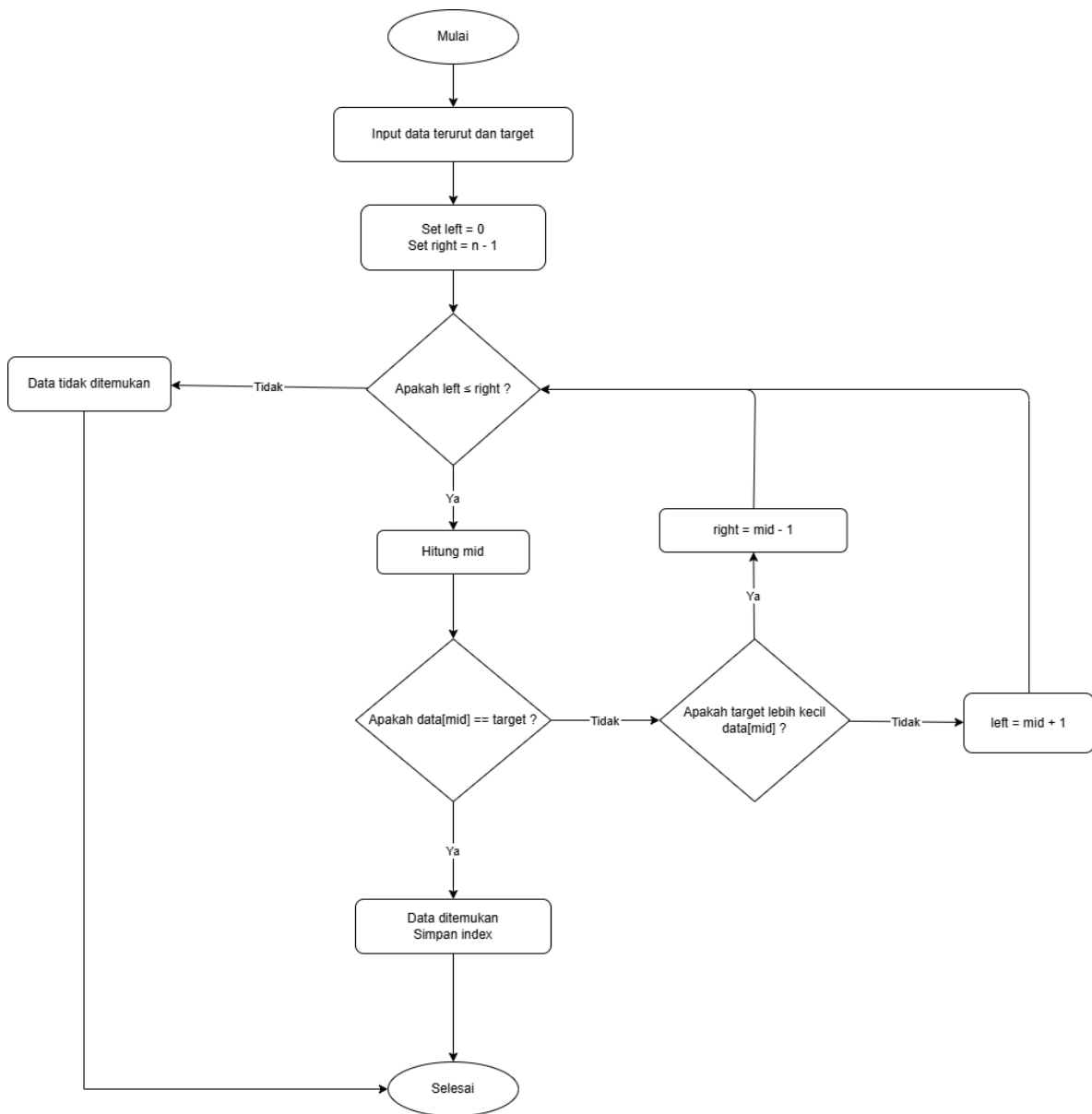
Berdasarkan analisis operasi, jumlah comparison pada Binary Search jauh lebih sedikit dibanding Linear Search. Pada data berukuran 1000000, Binary Search hanya membutuhkan sekitar 20 comparison pada worst case, sedangkan Linear Search membutuhkan hingga 1000000 comparison. Hal ini menunjukkan efisiensi Binary Search pada dataset besar.

Pada algoritma ini implementasi yang digunakan bersifat iteratif sehingga tidak terdapat recursion call. Jumlah array accessed mengikuti jumlah comparison karena setiap iterasi hanya mengakses elemen tengah array.

Dari hasil eksperimen didapatkan bahwa ukuran data tidak terlalu mempengaruhi waktu eksekusi Binary Search. Waktu eksekusi hampir selalu berada di sekitar 1–4 mikrosecond meskipun ukuran data meningkat hingga satu juta elemen.

Hasil ini menunjukkan bahwa Binary Search jauh lebih efisien dibanding Linear Search terutama pada dataset besar. Semakin besar ukuran data, perbedaan performa kedua algoritma menjadi semakin jelas. Binary Search mampu mempertahankan jumlah comparison yang kecil karena area pencarian selalu dibagi dua.

Dengan demikian, hasil eksperimen yang diperoleh sesuai dengan teori kompleksitas Binary Search yaitu $O(\log n)$ pada average case dan worst case. Dari praktikum ini dapat disimpulkan bahwa Binary Search sangat cocok digunakan untuk dataset besar yang sudah terurut karena mampu memberikan performa pencarian yang jauh lebih cepat dan efisien dibanding Linear Search.



Gambar 4 Flowchart Binary Search

TAUTAN GIT

<https://github.com/DSA26-ULM/module-5-searching-RyuuZev>